GNU Compiler Collection (gcc)

Pasos de Compilación, Visualización Intermedia y Optimización

Del Preprocesamiento al Código Ejecutable usando GCC

Resumen: Esta guía tiene como objetivo principal enseñar a los estudiantes de ingeniería a desglosar el proceso completo de compilación con GCC, desde el código fuente en C hasta el ejecutable final. A través de ejemplos prácticos reproducibles, mediciones cuantitativas y visualizaciones, el lector aprenderá a inspeccionar cada etapa intermedia (preprocesado, ensamblador, objeto), comprender el impacto real de diferentes niveles de optimización (-O0 a -O3, -Os) en el tamaño del código y el tiempo de ejecución, y aplicar mejores prácticas para proyectos universitarios y profesionales.

Curso: Sistemas Operativos

German Sánchez Torres, Ph.D.

Versión: 1.0

DRAFT

Índice general

1 Introducción **5**
1.1 Objetivos de la Guía 5
1.2 Público Objetivo 5
1.3 Prerrequisitos 6
1.3.1 Conocimientos de Programación 6
1.3.2 Entorno de Trabajo 6
1.3.3 Hardware y Software Mínimo 6
1.4 Metodología 6
1.4.1 Aprendizaje Basado en Ejemplos 7
1.4.2 Progresión Estructurada 7
1.5 Estructura General de la Guía 7
1.6 Recursos Adicionales 8
1.6.1 Documentación Oficial 8
1.6.2 Herramientas Online 8
1.6.3 Libros Recomendados 8

2 Herramientas Avanzadas y Depuración **9**
2.1 Depuración con GDB 9
2.1.1 Fundamentos de la Depuración 9
2.1.2 Compilación con Información de Debug 10
2.1.3 Introducción a GDB 10
2.1.4 Comandos Principales de GDB 11
2.1.5 Flujo de Trabajo de Depuración 12
2.1.6 Ejemplo de Sesión GDB Completa 12
2.2 Opciones Avanzadas de GCC 14
2.2.1 -fverbose-asm 14
2.2.2 -fdump-tree-all 14
2.2.3 -fopt-info 14
2.3 Compiler Explorer (godbolt.org) 15
2.3.1 Descripción 15
2.3.2 Uso Básico 15
2.4 Profiling con perf 15
2.4.1 Instalación 15
2.4.2 Comandos Básicos de perf 16
2.4.3 Ejemplo de perf stat 16
2.4.4 Flame Graphs 16
2.5 Profiling con gprof 17
2.5.1 Compilación para gprof 17
2.5.2 Interpretación de Resultados 17
2.6 Análisis Estático 17
2.6.1 Opciones de GCC para Análisis 17

DRAFT

2.6.2 Herramientas Externas 18
2.7 Valgrind para Análisis de Memoria 18
2.7.1 Detección de Fugas de Memoria 18
2.7.2 Ejemplo de Salida de Valgrind 18
2.8 Resumen del Capítulo 19

DRAFT

Índice de figuras

2.1 Flujo de trabajo de depuración con GDB. 13

Índice de cuadros

1.1 Requisitos mínimos del sistema 6

DRAFT

Capítulo 1

Introducción

1.1 Objetivos de la Guía

Esta guía tiene como propósito fundamental proporcionar a los estudiantes de ingeniería en sistemas y afines a la computación una comprensión profunda y práctica del proceso de compilación utilizando el GNU Compiler Collection (GCC). Los objetivos específicos son:

1. **Entender el flujo completo de compilación de GCC:** Desde el código fuente en C hasta el archivo ejecutable final, comprendiendo cada etapa del proceso y su importancia en el desarrollo de software.
2. **Aprender a inspeccionar etapas intermedias:** Dominar las técnicas para visualizar y analizar el código preprocesado, el ensamblador generado y los archivos objeto, desarrollando habilidades de depuración a bajo nivel.
3. **Analizar el impacto de optimizaciones:** Evaluar cuantitativamente cómo diferentes niveles de optimización (-O0, -O1, -O2, -O3, -Os) afectan el tamaño del código binario y el tiempo de ejecución, permitiendo tomar decisiones informadas en proyectos reales.

1.2 Público Objetivo

Esta guía está diseñada específicamente para:

- **Estudiantes de 3er semestre o superior** de Ingeniería en Sistemas Computacionales, Ingeniería en Computación o carreras afines.
- **Desarrolladores junior** que desean profundizar en el entendimiento de las herramientas de compilación.
- **Entusiastas de la programación de bajo nivel** interesados en comprender la relación entre código C y ensamblador.
- **Investigadores académicos** que necesitan analizar el rendimiento de algoritmos a nivel de código máquina.

DRAFT

Aunque la guía está orientada a estudiantes de nivel intermedio, los conceptos se presentan de manera progresiva, permitiendo que incluso estudiantes de semestres iniciales puedan seguir el material con dedicación.

1.3 Prerrequisitos

Para aprovechar al máximo esta guía, se recomienda contar con los siguientes conocimientos previos:

1.3.1 Conocimientos de Programación

- **Lenguaje C:** Familiaridad con sintaxis básica, estructuras de control (if, for, while), funciones, punteros y manejo básico de memoria.
- **Conceptos de algoritmos:** Comprensión de complejidad temporal y espacial básica.
- **Programación estructurada:** Capacidad para escribir y leer código modular.

1.3.2 Entorno de Trabajo

- **Terminal/Linux:** Conocimientos básicos de navegación por directorios, ejecución de comandos y redirección de salida.
- **Editor de texto:** Capacidad para editar archivos de código (recomendado: VS Code, Vim, o Nano).
- **Git (opcional):** Para clonar repositorios de ejemplos.

1.3.3 Hardware y Software Mínimo

Tabla 1.1: Requisitos mínimos del sistema

Componente	Requisito
Sistema Operativo	Linux (Ubuntu 20.04+), WSL2, o macOS
Memoria RAM	4 GB mínimo (8 GB recomendado)
Espacio en disco	2 GB libres
Procesador	x86_64 (Intel/AMD) o ARM64
GCC	Versión 9.0 o superior

1.4 Metodología

Esta guía adopta un enfoque **práctico y experimental**, fundamentado en los siguientes principios pedagógicos:

DRAFT

1.4.1 Aprendizaje Basado en Ejemplos

Cada concepto teórico se acompaña de ejemplos de código reproducibles. El lector puede ejecutar todos los comandos y programas presentados en su propio entorno.

1.4.2 Progresión Estructurada

El contenido se organiza de lo simple a lo complejo:

1. **Fundamentos:** Comprensión básica de GCC y sus componentes
2. **Descomposición:** Análisis detallado de cada paso de compilación
3. **Optimización:** Experimentación con diferentes niveles de optimización
4. **Herramientas avanzadas:** Uso de depuradores y perfiladores
5. **Proyectos prácticos:** Aplicación integrada de conocimientos

1.5 Estructura General de la Guía

Esta guía se organiza en siete capítulos principales, más anexos con soluciones y referencias:

Capítulo 1: Introducción
Contexto, objetivos y metodología de la guía (este capítulo).

Capítulo 2: Fundamentos de GCC
Historia, instalación, componentes y opciones básicas del compilador.

Capítulo 3: Los Pasos de Compilación
Desglose detallado del flujo completo: preprocesamiento, generación de ensamblador, ensamblado y enlace.

Capítulo 4: Niveles de Optimización
Análisis exhaustivo de -O0, -O1, -O2, -O3, -Os, con experimentos cuantitativos y visualizaciones.

Capítulo 5: Herramientas Avanzadas
Depuración con GDB, profiling con perf, y otras utilidades profesionales.

Capítulo 6: Ejercicios y Prácticas
Problemas graduados de dificultad básica, intermedia y avanzada.

Capítulo 7: Conclusión
Resumen de aprendizajes, mejores prácticas y limitaciones de GCC.

Se recomienda seguir los capítulos en orden secuencial, ya que cada uno construye sobre los conceptos del anterior. Sin embargo, los capítulos 5 y 6 pueden consultarse de manera independiente como referencia.

DRAFT

1.6 Recursos Adicionales

Para complementar el aprendizaje, se recomiendan los siguientes recursos:

1.6.1 Documentación Oficial

- **GCC Online Documentation:** https://gcc.gnu.org/onlinedocs/
- **GNU Binutils:** https://sourceware.org/binutils/docs/
- **GDB Manual:** https://sourceware.org/gdb/current/onlinedocs/gdb/

1.6.2 Herramientas Online

- **Compiler Explorer (godbolt.org):** Visualización interactiva de ensamblador generado por múltiples compiladores.
- **Godbolt Compiler Explorer:** https://godbolt.org

1.6.3 Libros Recomendados

- *Computer Systems: A Programmer's Perspective* (Bryant & O'Hallaron)
- *The Art of Assembly Language* (Randall Hyde)
- *Linkers and Loaders* (John R. Levine)

Los enlaces a recursos externos pueden cambiar. Si un enlace no funciona, se recomienda buscar el recurso por nombre en un motor de búsqueda.

DRAFT

Capítulo 2

Herramientas Avanzadas y Depuración

Este capítulo explora herramientas profesionales que complementan el uso de GCC para el desarrollo, depuración y análisis de rendimiento de aplicaciones.

2.1 Depuración con GDB

2.1.1 Fundamentos de la Depuración

¿Qué es Depurar Código?

La depuración (*debugging*) es el proceso sistemático de buscar, identificar y corregir errores (*bugs*) en un programa. Es una habilidad fundamental, especialmente en lenguajes de bajo nivel como C, donde la memoria y la ejecución deben gestionarse explícitamente.

Tipos de Errores en C

En el desarrollo de programas en C, es crucial distinguir entre diferentes categorías de errores:

- **Errores de compilación:** errores de sintaxis o semántica detectados por el compilador (por ejemplo, falta de punto y coma o variables no declaradas).
- **Errores de ejecución:** fallos que ocurren mientras el programa se está ejecutando (como divisiones por cero o violaciones de segmento).
- **Errores lógicos:** el programa se ejecuta pero produce resultados incorrectos debido a fallos en el algoritmo o la lógica implementada.
- **Errores de memoria:** acceso inválido a memoria, fugas (*memory leaks*) o corrupción de datos (particularmente peligrosos en C debido a la gestión manual de memoria).

Los errores de memoria en C pueden manifestarse silenciosamente durante la ejecución y causar fallos catastróficos en producción. Herramientas como Valgrind (Sección 2.7) complementan a GDB para detectar este tipo de problemas.

DRAFT

2.1.2 Compilación con Información de Debug

Para depurar de manera efectiva, el ejecutable debe incluir símbolos de depuración y compilarse sin optimizaciones agresivas que reordenen el código:

```
# Compilacion recomendada para debugging
gcc -g -Wall -Wextra -O0 programa.c -o programa

# Desglose de flags:
# -g: incluye simbolos de depuracion
# -O0: desactiva la optimizacion del compilador (mantiene correspondencia 1:1
    con el codigo fuente)
# -Wall -Wextra: muestra advertencias utiles sobre codigo sospechoso
```

Código 2.1: Compilacion con simbolos de debug

Los niveles de información de debug disponibles son:

```
gcc -g0 # Sin informacion (por defecto en -O1+)
gcc -g1 # Informacion minima (solo funciones globales)
gcc -g # Informacion completa (equivalente a -g2)
gcc -g3 # Informacion completa + macros (maximo detalle)
```

Código 2.2: Niveles de informacion de debug

La optimización (`-O2`, `-O3`) puede reordenar código, eliminar variables temporales e inlinear funciones, haciendo que el debugging sea confuso. Para debugging completo, use siempre `-O0 -g`. Solo combine `-g` con optimización si necesita depurar código optimizado específicamente.

2.1.3 Introducción a GDB

GDB (GNU Debugger) es una potente herramienta de línea de comandos utilizada para depurar programas escritos en C y C++, permitiéndote monitorear y controlar la ejecución del código paso a paso.

Inicio de una Sesión GDB

Una vez compilado el programa con las banderas de depuración, inicia GDB:

```
# Iniciar GDB con el ejecutable
gdb ./programa

# Deberias ver el prompt de GDB:
(gdb)
```

Código 2.3: Inicio de GDB

Para comenzar la ejecución dentro del entorno:

```
(gdb) run # Ejecuta el programa hasta el primer breakpoint o el final
```

```
(gdb) run arg1 arg2 # Ejecuta con argumentos de linea de comandos
```

2.1.4 Comandos Principales de GDB

Gestión de Breakpoints

Un *breakpoint* pausa la ejecución en un punto específico, permitiendo inspeccionar el estado del programa:

```
(gdb) break main # En la funcion main
(gdb) break 25 # En la linea 25 del archivo actual
(gdb) break funcion # En una funcion especifica
(gdb) break archivo.c:42 # En linea 42 de archivo especifico
(gdb) info breakpoints # Listar breakpoints activos
(gdb) delete 1 # Eliminar breakpoint numero 1
```

Código 2.4: Comandos de breakpoints

Navegación Paso a Paso: Next vs Step

La diferencia entre estos comandos es fundamental para una navegación eficiente:

```
(gdb) next # Ejecuta la siguiente linea (no entra a funciones)
(gdb) step # Ejecuta la siguiente linea (entra dentro de funciones)
(gdb) continue # Continua hasta el siguiente breakpoint o el final
(gdb) finish # Ejecuta hasta salir de la funcion actual
```

Código 2.5: Navegacion paso a paso

Regla práctica:

- Usa `next` para moverte rápidamente por código que sabes que funciona correctamente.
- Usa `step` cuando sospeches que el bug está dentro de una llamada a función específica.

Inspección de Variables

```
(gdb) print variable # Muestra valor de variable
(gdb) p variable # Forma abreviada
(gdb) p *ptr # Inspecciona a que hace referencia un puntero
(gdb) p my_struct # Muestra contenido de estructuras
(gdb) p/x variable # Muestra en hexadecimal
(gdb) p/d variable # Muestra en decimal (por defecto)
(gdb) display variable # Muestra valor automaticamente en cada paso
(gdb) info locals # Muestra todas las variables locales
```

Código 2.6: Comandos de inspeccion

Análisis de Pila de Llamadas

El *backtrace* muestra la pila de llamadas (*stack trace*), especialmente útil cuando ocurre un *crash*:

```
(gdb) backtrace # Muestra la pila de llamadas completa
(gdb) bt # Forma abreviada
(gdb) bt full # Incluye valores de variables locales en cada frame
(gdb) frame 2 # Cambia al frame numero 2 de la pila
(gdb) up # Sube un nivel en la pila
(gdb) down # Baja un nivel en la pila
```

Código 2.7: Backtrace

2.1.5 Flujo de Trabajo de Depuración

Un enfoque sistemático es esencial para depurar eficientemente (ver Figura 2.1):

1. **Compilar con banderas de depuración:** usar `gcc -g -O0 -Wall -Wextra`.
2. **Lanzar GDB:** ejecutar `gdb ./programa`.
3. **Establecer breakpoints:** ubicarlos en funciones sospechosas o áreas problemáticas.
4. **Ejecutar el programa:** usar `run` para iniciar la ejecución controlada.
5. **Avanzar paso a paso:** utilizar `next` o `step` según sea necesario.
6. **Inspeccionar variables y memoria:** verificar valores con `print` y estado de punteros.
7. **Analizar el backtrace:** si ocurre un fallo, usar `bt` para identificar la secuencia de llamadas.
8. **Corregir y repetir:** modificar el código, recompilar y repetir el proceso hasta resolver el bug.

2.1.6 Ejemplo de Sesión GDB Completa

El siguiente ejemplo ilustra una sesión típica de debugging:

```
$ gcc -g -Wall -O0 -o programa programa.c
$ gdb ./programa
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
Reading symbols from ./programa...
(gdb) break main
Breakpoint 1 at 0x1149: file programa.c, line 5.
(gdb) run
Starting program: ./programa

Breakpoint 1, main () at programa.c:5
5 int x = 10;
(gdb) next
6 int y = x * 2;
(gdb) print x
$1 = 10
(gdb) next
```

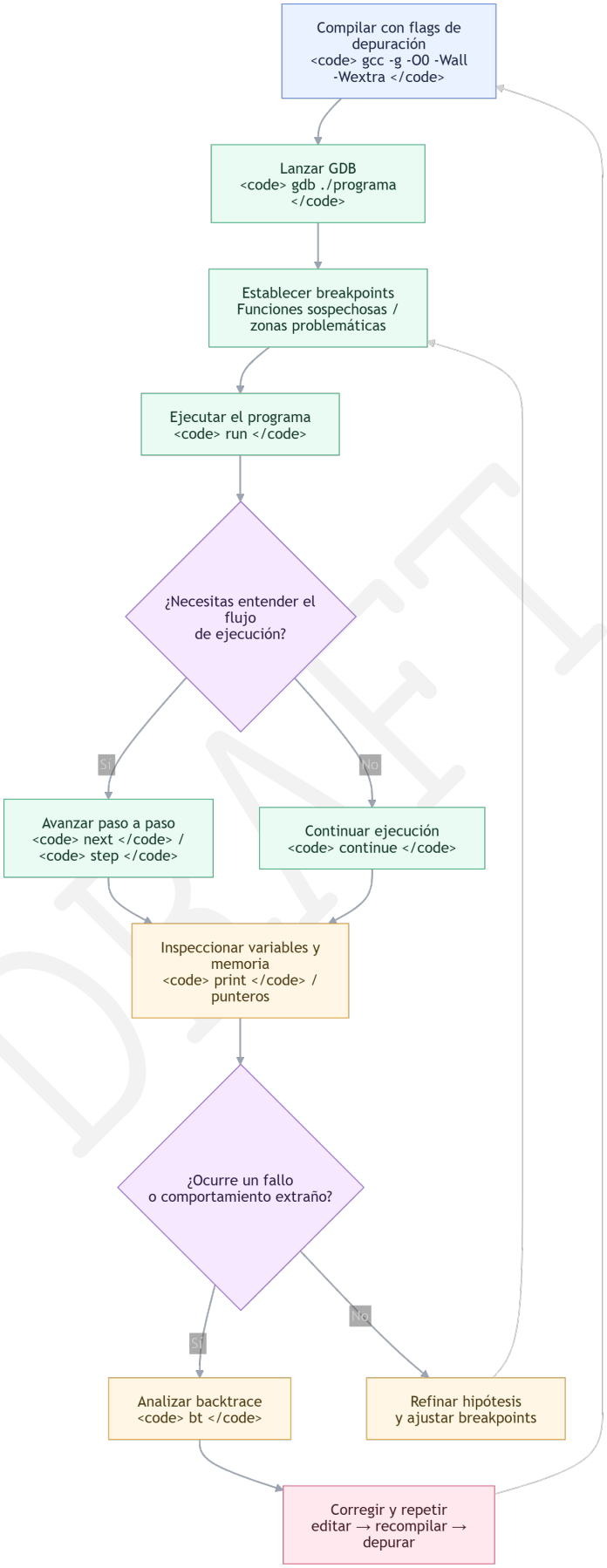

Figura 2.1: Flujo de trabajo de depuración con GDB.

```
7 printf("Resultado: %d\n", y);
(gdb) print y
$2 = 20
(gdb) continue
Resultado: 20
[Inferior 1 (process 12345) exited normally]
(gdb) quit
```

Código 2.8: Ejemplo completo de sesion GDB

2.2 Opciones Avanzadas de GCC

2.2.1 -fverbose-asm

Genera código ensamblador con comentarios que mapean a líneas de C:

```
gcc -S -fverbose-asm -O2 programa.c

# El archivo .s incluira comentarios como:
# .loc 1 15 # Linea 15 del archivo fuente
```

Código 2.9: Ensamblador verboso

2.2.2 -fdump-tree-all

Volcado de representaciones intermedias del compilador:

```
# Generar dumps de todas las fases
gcc -fdump-tree-all -O2 programa.c

# Genera archivos como:
# programa.c.003t.original # Arbol original
# programa.c.005t.gimple # Representacion GIMPLE
# programa.c.010t.eh # Despues de manejo de excepciones
# programa.c.020t.ssa # Forma SSA
# programa.c.100t.optimized # Arbol optimizado
```

Código 2.10: Volcado de arboles de optimizacion

2.2.3 -fopt-info

Información sobre optimizaciones aplicadas:

```
# Ver optimizaciones aplicadas
gcc -O2 -fopt-info programa.c

# Guardar a archivo
gcc -O2 -fopt-info-vec-optimized=vec.log programa.c

# Ver vectorizaciones fallidas
gcc -O3 -fopt-info-vec-missed=missed.log programa.c
```

Código 2.11: Informacion de optimizaciones

2.3 Compiler Explorer (godbolt.org)

2.3.1 Descripción

Compiler Explorer (https://godbolt.org) es una herramienta web que permite:

- Visualizar código ensamblador generado en tiempo real
- Comparar múltiples compiladores (GCC, Clang, MSVC, etc.)
- Probar diferentes niveles de optimización
- Compartir ejemplos mediante URLs
- Analizar el impacto de cambios específicos

2.3.2 Uso Básico

1. Escribir o pegar código C en el panel izquierdo
2. Seleccionar compilador y flags en el menú superior
3. El ensamblador se muestra automáticamente en el panel derecho
4. Las líneas de C y ensamblador están coloreadas para correlación

Compiler Explorer es invaluable para aprender sobre optimizaciones. Puede ver inmediatamente cómo cambia el código generado al modificar el nivel de optimización o la arquitectura objetivo.

2.4 Profiling con perf

2.4.1 Instalación

```
# Ubuntu/Debian
sudo apt install linux-tools-common linux-tools-generic

# Fedora
sudo dnf install perf

# Verificar
perf --version
```

Código 2.12: Instalacion de perf

2.4.2 Comandos Básicos de perf

```
# Grabar estadisticas de ejecucion
perf record ./programa

# Analizar resultados
perf report

# Estadisticas en tiempo real
perf stat ./programa

# Perf stat detallado
perf stat -e cycles,instructions,cache-misses ./programa

# Anotar codigo fuente con estadisticas
perf annotate
```

Código 2.13: Uso basico de perf

2.4.3 Ejemplo de perf stat

```
$ perf stat ./programa

 Performance counter stats for './programa':

        1,234.56 msec task-clock # 0.999 CPUs utilized
             123 context-switches # 0.100 K/sec
               1 cpu-migrations # 0.001 K/sec
             456 page-faults # 0.370 K/sec
   3,456,789,012 cycles # 2.801 GHz
   5,678,901,234 instructions # 1.64 insn per cycle
     123,456,789 branches # 100.000 M/sec
         123,456 branch-misses # 0.10% of all branches

      1.23567890 seconds time elapsed
```

Código 2.14: Salida de perf stat

2.4.4 Flame Graphs

```
# Grabar con call graph
perf record -g ./programa

# Generar reporte
perf script | ./stackcollapse-perf.pl | ./flamegraph.pl > perf.svg

# Abrir en navegador
firefox perf.svg
```

Código 2.15: Generacion de Flame Graphs

2.5 Profiling con gprof

2.5.1 Compilación para gprof

```
# Compilar con soporte para profiling
gcc -pg -o programa programa.c

# Ejecutar (genera gmon.out)
./programa

# Analizar resultados
gprof ./programa gmon.out

# Salida plana (flat profile)
gprof -p ./programa gmon.out

# Grafo de llamadas
gprof -q ./programa gmon.out
```

Código 2.16: Compilacion para gprof

2.5.2 Interpretación de Resultados

```
Flat profile:

Each sample counts as 0.01 seconds.
  % cumulative self self total
 time seconds seconds calls ms/call ms/call name
 45.0 0.45 0.45 100000 0.00 0.00 funcion_costosa
 30.0 0.75 0.30 500000 0.00 0.00 otra_funcion
 15.0 0.90 0.15 1000000 0.00 0.00 funcion_simple
 10.0 1.00 0.10 main
```

Código 2.17: Ejemplo de salida gprof

2.6 Análisis Estático

2.6.1 Opciones de GCC para Análisis

```
# Advertencias exhaustivas
gcc -Wall -Wextra -Wpedantic programa.c

# Advertencias como errores
gcc -Wall -Werror programa.c

# Analisis de comportamiento indefinido
gcc -Wstrict-overflow -Wstrict-aliasing programa.c

# Analisis de formatos
gcc -Wformat=2 -Wformat-security programa.c
```

Código 2.18: Opciones de analisis estatico

2.6.2 Herramientas Externas

```
# cppcheck - Analizador estatico
sudo apt install cppcheck
cppcheck --enable=all programa.c

# clang-static-analyzer
scan-build gcc -o programa programa.c

# Valgrind (analisis dinamico)
valgrind --leak-check=full ./programa
valgrind --tool=cachegrind ./programa
```

Código 2.19: Herramientas de analisis estatico

2.7 Valgrind para Análisis de Memoria

2.7.1 Detección de Fugas de Memoria

```
# Detectar fugas de memoria
valgrind --leak-check=full ./programa

# Con informacion de donde se asigno
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./programa

# Perfilado de cache
valgrind --tool=cachegrind ./programa

# Perfilado de llamadas
valgrind --tool=callgrind ./programa
kcachegrind callgrind.out.*
```

Código 2.20: Uso de Valgrind

2.7.2 Ejemplo de Salida de Valgrind

```
$ valgrind --leak-check=full ./programa
==12345== Memcheck, a memory error detector
==12345==
==12345== HEAP SUMMARY:
==12345== in use at exit: 0 bytes in 0 blocks
==12345== total heap usage: 10 allocs, 10 frees, 1,000 bytes allocated
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

Código 2.21: Salida tipica de Valgrind

Valgrind es una herramienta esencial para detectar errores de memoria que pueden pasar desapercibidos pero causar fallos en producción. Úselo regularmente durante el desarrollo.

2.8 Resumen del Capítulo

En este capítulo hemos explorado:

- Depuración con GDB a nivel de código fuente y ensamblador
- Opciones avanzadas de GCC para análisis (-fverbose-asm, -fdump-tree-all)
- Compiler Explorer para visualización interactiva
- Profiling de rendimiento con perf y gprof
- Análisis estático con opciones de GCC y herramientas externas
- Detección de errores de memoria con Valgrind

Estas herramientas forman el arsenal profesional del desarrollador de C/C++ y son esenciales para crear software robusto y de alto rendimiento.

DRAFT